

IDS Alert Prioritization and Strengthening

Nathan Elderbroom, Gabriel Goetten, Oskari Pirhonen, Abhishek Thurlapati

Introduction

Intrusion Detection Systems monitor network traffic and provide alerts about potentially malicious network traffic. A big problem with these detection systems is preventing false positives. Usually these systems take the approach to make their systems alert too often, because it is easier to deal with false positives rather than accidentally let malware slip through. One potential solution to this would be to use a neural network to classify network traffic, potentially reducing the number of false positives. There are also several options within neural networks that we will investigate.

Plan

1. Analyze datasets and find correlations between attribute-values.
2. Figure out the best method to train the data
3. Experimenting with different neural network structures and designs (Simple, Recurrent)
4. Implement ML methodologies to train further to improve on accuracy
5. Repeat the process several times to enhance the final ML algorithm to be as accurate as possible without overfitting the data

The goal is to train a neural network to classify packets. Using the neural network to prioritize alerts from a more traditional IDS system.

Schedule

Proposal - February 14
Investigate Datasets - Feb 21
Midterm (progress) report - April 3
Midterm presentation - April 3
Refine Approach - Apr 24
Final deliverables - May 1
Final presentation - May 1
Mentor Meetings - Every Friday

Progress

We decided to use the CSE-CIC-IDS2018 dataset to do our research. After learning a bit of the domain knowledge of our dataset, we first decided to find the importance of each column on the outcome (benign/malicious). From there we decided on certain columns to drop, either because they had problems or because our analysis showed very little correlation to benign/malicious. We then looked to fill in any missing values to make sure all data was present. Due to the dataset being so large, we decided to scale it down into eight separate csv files. These changes helped significantly reduce testing times and enabled us to be able to run more trials. Running visuals such as PCA scatterplot and correlation plots, we noticed that the attacks are centered mostly within the benign data points. We used these visuals to gain a better understanding on how we should further tweak our dataset and run our model to classify the attacks versus the benign instances.

We started with a sequential model from the Keras library. This is a simple linear model with 2 layers of 64 nodes each. We later expanded this to 128 nodes each. We decided to use Keras because it allows us to change the model as needed later down the road. Keras itself is pretty high-level, but due to its integration with tensorflow, we can use low-level features and abilities if we want to use a more complex model later. Its flexibility allows us to improve/change our neural net model. The neural net training technique was ideal for the way our data was distributed. There was no perfect distinction between the attack and benign points, so the versatility of neural nets will be able to better classify our data.

After getting a basic neural network setup, we decided to also set up a Recurrent Neural Network so that we could compare the two different strategies. We investigated the recurrent neural network because it could potentially draw more conclusions about relationships between several different data points. In contrast, our simpler model only considered individual data points. We also decided to look at two different methods of classifying the malware. We considered both a binary classification problem, sorting the network traffic into simply “Benign” or “Malicious”, and a more complex version of the problem, sorting the malware into more specific malware categories. We used both methods of classification in both simple and recurrent networks, looking to compare the results of each.

Due to the fact that a very large portion of our dataset was benign, we occasionally ran into issues where the model was getting a large amount of its accuracy from simply classifying all data as benign. This is not a useful result. We quickly realized that we needed to weight the malicious data higher to make it have an equal impact on training to the benign data. This lessened the effect of the data imbalance.

Results

The biggest comparison we were looking to make was the simple neural network versus the Recurrent Neural Network. This comparison isn’t quite an equal one, as the recurrent model is inherently more complex. This meant that the recurrent model was considerably slower to train and to use after the fact. The performance drawback did come with benefits however. We found that after training both models for the same number of epochs, the recurrent network had an accuracy that was slightly better.

We also found the binary classification problems seemed to perform better than trying to classify the malware into a specific category. On our recurrent model, we had difficulty getting any progress when trying to classify the malware into specific types. While knowing the specific type of malware is a useful feature, it may not always be necessary, and drastically increases the complexity and execution times of the network. Splitting the classification into several different malware subtypes also splits up the dataset, providing fewer data points and furthering the imbalance for any specific classification task.

Overall, we think all the models that we were attempting could produce better results if given more time and compute power. We ran into time limitations running code on Google Colab to train our models.

Conclusion

Our results showed that Recurrent Neural Network Models may be worth pursuing for Intrusion Detection Systems, however it only performed slightly better than the simpler neural networks we were using. We also found that binary classification of Benign or Malware worked better than

classifying network traffic into one of several different types of malware.

Related work

Using Adaptive Alert Classification to Reduce False Positives in Intrusion Detection

This paper appears to be one of the earlier papers to attempt using machine learning to solve the problem of false positive alerts. The model uses a neural network and machine learning to classify attacks by whether or not it is a false positive, and requires an analyst to correctly audit and adjust the program's results to train the data. The paper goes in depth about how crucial it is to provide proper background knowledge about the system to the neural network, to allow it to 'learn' relevant information by classifying certain data before it is fed to the neural network.

Data Mining and Machine Learning: Towards reducing false positives in Intrusion Detection

This paper presents two different, but orthogonal approaches to dealing with the problem of false positives. The first approach is to cluster many similar alerts together based on a root cause, so that a human reading the alerts has fewer total alerts to sort through. This is a pretty common method of dealing with too many alerts. The paper also uses an additional machine-learning program to either add labels to, or to directly filter the incoming alerts to reduce the number of alerts. The machine learning system essentially attempts to label alerts as 'accurate' or 'false positive' by automated means. These two approaches, clustering and filtering, work in tandem to reduce false positives.

Finding Needles in a Moving Haystack: Prioritizing Alerts with Adversarial Reinforcement Learning

This paper details an interesting approach to prioritizing alerts for intrusion detection systems. The approach involves using Adversarial Reinforcement learning to simulate both a defender and an adversary. This approach seems like it has some potential, because the defender/adversary approach mimics what happens in reality, with an IDS system needing to handle attacks from an adversary, which may be able to change their attack method. In the simulated scenario, the attacker can adjust the likelihood of different attack methods, and the defender must try to accurately prevent false positives in varying conditions.

Alert Fusion for A Computer Host Based Intrusion Detection System

This paper takes a pretty interesting approach to managing the IDS system alerts. The authors have essentially hand-crafted a neural network to determine the alert priority. They base their customized neural network on a "STRIDE/DREAD" threat model, with nodes hand-weighted for certain instances. This model doesn't involve machine learning, but the authors mention that in future, backpropagation can be used to 'train' the network for further improvements. This method is very interesting because a hand-designed setup like this can be understood directly, and tweaked further with ease to adjust the severities of specific types of attacks.

Deep Learning for Prioritizing and Responding to Intrusion Detection Alerts

The paper discusses the results of the Federal Analysis Security Triage Tool (FASTT), which

relies on a machine learning approach, employing Deep Neural Network (DNN) classifiers to automatically categorize IDS alerts. The system consists of an IDS and a data pipeline, which retrieves data from the IDS and classifies it using DNN classifiers. The results, recorded in an indexed database, are then presented to the appropriate security analyst through a graphical interface. Classifiers were trained using initial data labeled by security analysts. As security threats evolve, the authors note that the classifiers will improve with additional analyst feedback. The paper suggests future implementations using clustering method may lead to improvements in detection, allowing analysts to identify groupings that were overlooked through the heuristic approach.

A Hybrid High-Order Markov Chain Model for Computer Intrusion Detection

This paper goes over how to use a Markov chain to aid in intrusion detection. A simple Markov chain consists of an input and gives an output along with the probability of said output. In the paper, the inputs used are a series of Unix commands entered. A system utilizing this would have a baseline “signature” for each user in order to model how the user interacts with the command line. By comparing a session with the user’s signature, an IDS will be able to alert about potential anomalies in the commands entered. A similar system could potentially be expanded to targets such as network traffic. Typical traffic can be recorded and then modeled as a Markov chain using parameters such as typical packet size, connection length/frequency, and source/destination.

Combining Recurrence Quantification Analysis and Adaptive Clustering to Detect DDoS Attacks

This paper discusses using an adaptive algorithm to analyze network traffic and detect a DDoS attack which is more and more important as an increasing number of services rely on the Internet. The base algorithm, Recurrence Quantification Analysis (RQA), allows for mathematical analysis of a non-linear periodic signal to determine data points such as Recurrence Ratio, Determinism, and Entropy. To RQA, they add on what they call Adaptive Clustering (A-Kmeans) which can change the number of clusters used dynamically to better express the attack. This method, dubbed DDoSbyRQA, is an improvement on the traditional method simply analysing network traffic behavior based on a set of attributes. This older method has a major pitfall in that when normal traffic increases, so does the number of false positives.

References

- [1] Pietraszek, Tadeusz. "Using adaptive alert classification to reduce false positives in intrusion detection." *International Workshop on Recent Advances in Intrusion Detection*. Springer, Berlin, Heidelberg, 2004.
- [2] Pietraszek, Tadeusz, and Axel Tanner. "Data Mining and Machine learning—Towards Reducing False Positives in Intrusion Detection." *Information Security Technical Report* 10.3 (2005): 169–183. Web.
- [3] Liang, Tong et al. "Finding Needles in a Moving Haystack: Prioritizing Alerts with Adversarial Reinforcement Learning." *arXiv.org* (2019): n. pag. Web.
- [4] Chuan Feng et al. "Alert Fusion for a Computer Host Based Intrusion Detection System." *14th Annual IEEE International Conference and Workshops on the Engineering of Computer-Based Systems (ECBS'07)*. IEEE, 2007. 433–440. Web.

- [5] McElwee, Steven, et al. "Deep learning for prioritizing and responding to intrusion detection alerts." MILCOM 2017-2017 IEEE Military Communications Conference (MILCOM). IEEE, 2017.
- [6] Ju, Wen-Hua, and Yehuda Vardi. "A Hybrid High-Order Markov Chain Model for Computer Intrusion Detection." *Journal of Computational and Graphical Statistics*, vol. 10, no. 2, 2001, pp. 277–295. JSTOR, www.jstor.org/stable/1391012. Accessed 8 Feb. 2020.
- [7] Righi, Marcelo Antonio, and Raul Ceretta Nunes. "Combining Recurrence Quantification Analysis and Adaptive Clustering to Detect DDoS Attacks." *The Cyber Defense Review*, 2019, pp. 15–30. JSTOR, www.jstor.org/stable/26846118. Accessed 8 Feb. 2020.